

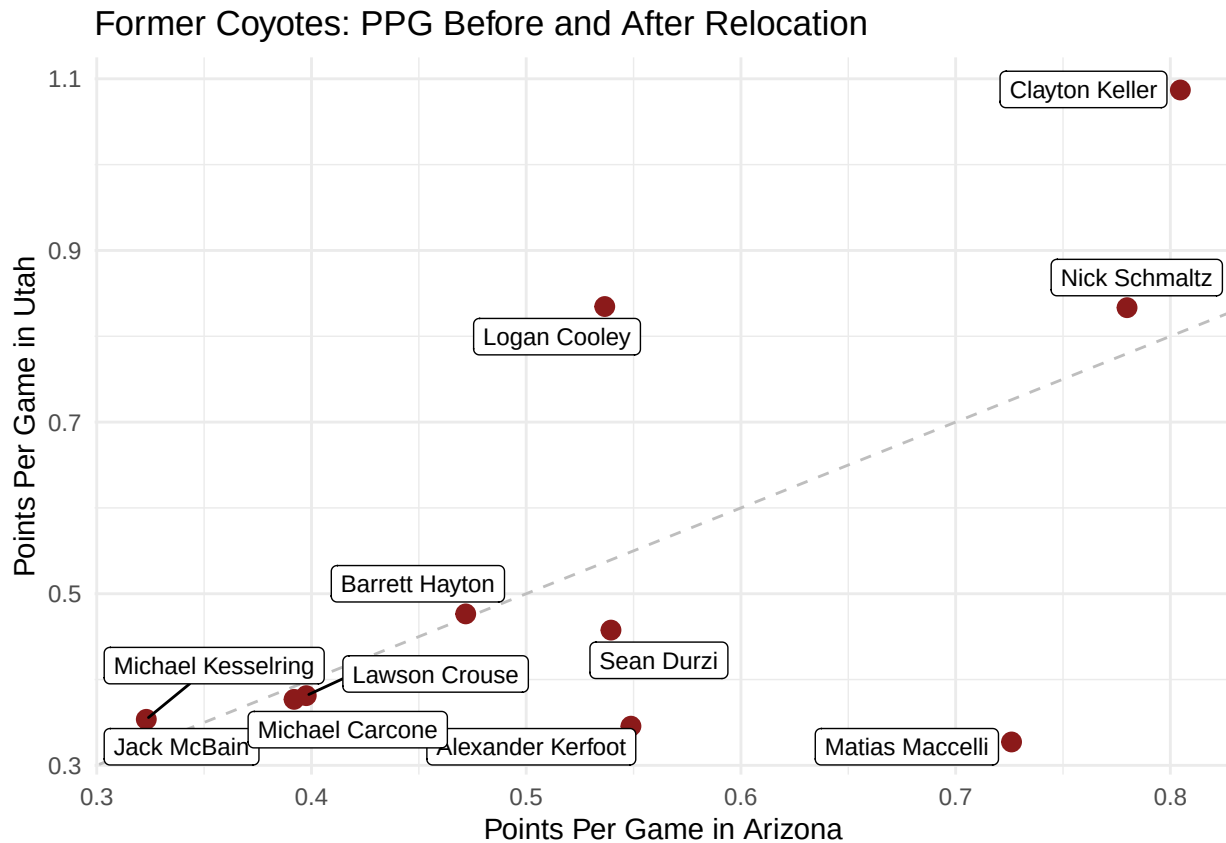
Appendix: Full R Code and Output

ARI vs UTA summary table

```
players_summary <- df %>%  
  filter(team %in% c("ARI", "UTA")) %>%  
  group_by(player_name, team) %>%  
  summarise(  
    avg_ppg = sum(points) / sum(gp),  
    .groups = "drop"  
  ) %>%  
  pivot_wider(  
    names_from = team,  
    values_from = avg_ppg  
  ) %>%  
  rename(  
    arizona_ppg = ARI,  
    utah_ppg = UTA  
  )
```

ARI vs UTA Plot

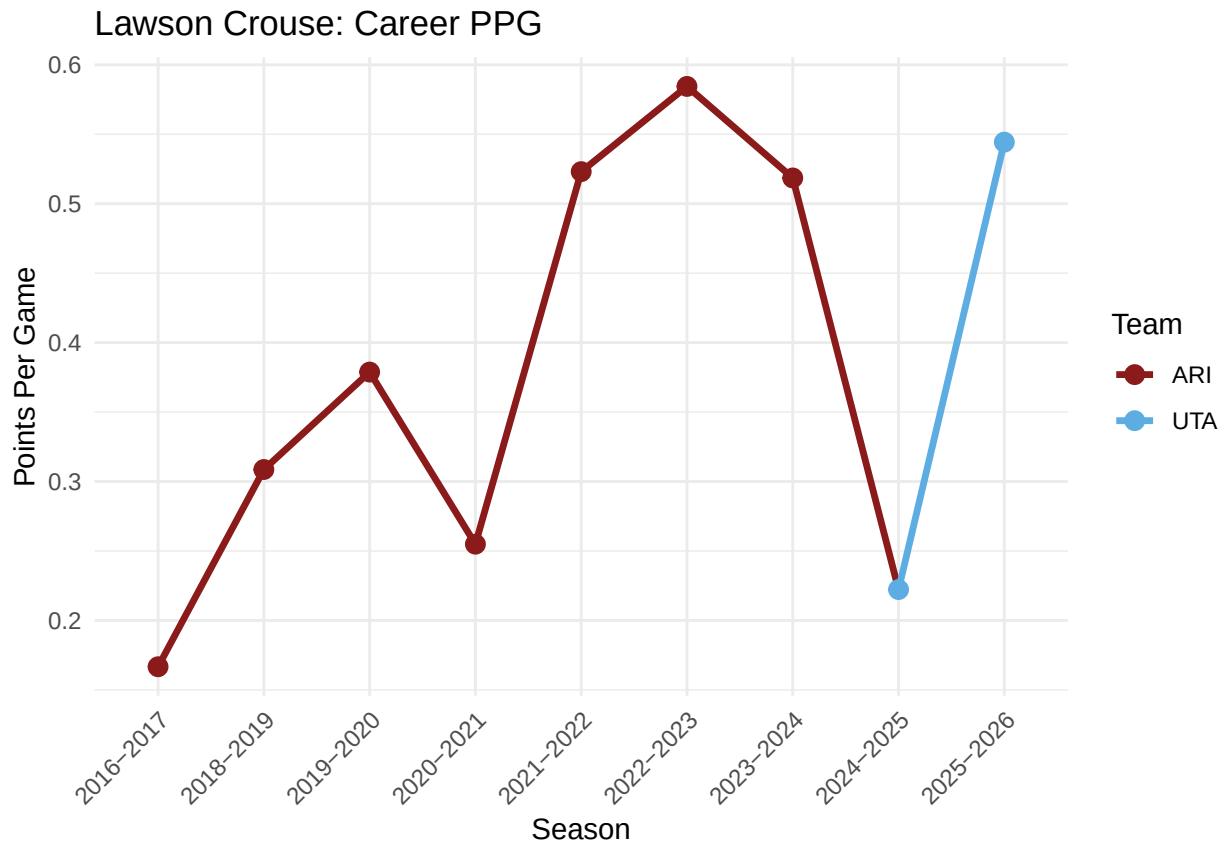
```
ggplot(players_summary, aes(x = arizona_ppg, y = utah_ppg, label = player_name)) +  
  geom_abline(slope = 1, intercept = 0, linetype = "dashed", color = "gray") +  
  geom_point(size = 3, color = "#8B1A1A") +  
  ggrepel::geom_label_repel(size = 3) +  
  labs(  
    title = "Former Coyotes: PPG Before and After Relocation",  
    x = "Points Per Game in Arizona",  
    y = "Points Per Game in Utah"  
  ) +  
  theme_minimal()
```



Crouse analysis

```
crouse_df <- df %>%
  filter(player_name == "Lawson Crouse") %>%
  arrange(season)
crouse_df <- crouse_df %>%
  mutate(season = factor(season, levels = unique(season)))

ggplot(crouse_df, aes(x = season, y = ppg, group = 1, color = team)) +
  geom_line(linewidth = 1.2) +
  geom_point(size = 3) +
  scale_color_manual(values = c("ARI" = "#8B1A1A", "UTA" = "#5DADE2")) +
  labs(
    title = "Lawson Crouse: Career PPG",
    x = "Season",
    y = "Points Per Game",
    color = "Team"
  ) +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```



Player overview bar chart

```
knitr::kable(players_summary,
  caption = "Average PPG: Arizona vs Utah Era",
  digits = 3)
```

Table 1: Average PPG: Arizona vs Utah Era

player_name	arizona_ppg	utah_ppg
Alexander Kerfoot	0.549	0.346
Barrett Hayton	0.472	0.477
Clayton Keller	0.805	1.087
Jack McBain	0.349	0.331
Lawson Crouse	0.398	0.381
Logan Cooley	0.537	0.835
Matias Maccelli	0.726	0.327
Michael Carcone	0.392	0.377
Michael Kesselring	0.323	0.354
Nick Schmaltz	0.780	0.833
Sean Durzi	0.539	0.458

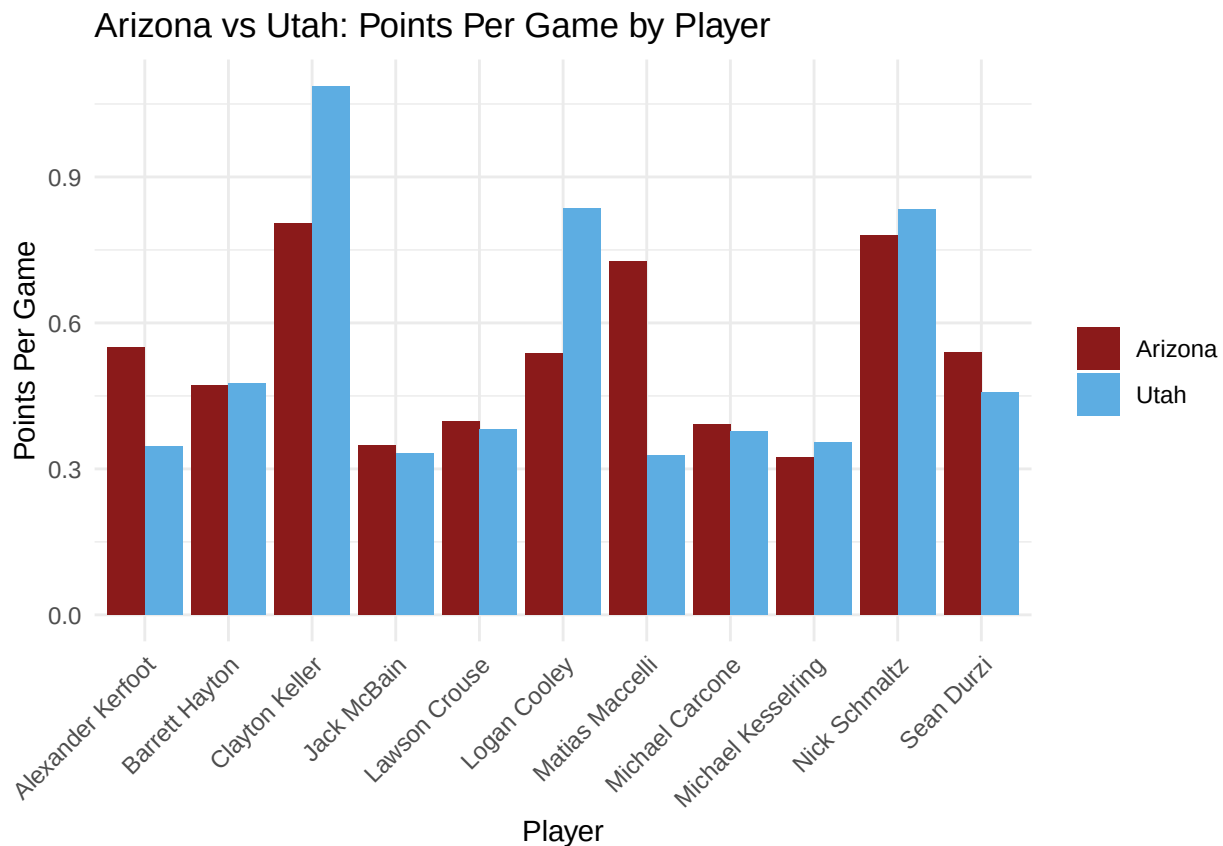
```
bar_df <- players_summary %>%
  pivot_longer(
    cols = c(arizona_ppg, utah_ppg),
    names_to = "team",
```

```

  values_to = "ppg"
) %>%
mutate(team = recode(team,
                      arizona_ppg = "Arizona",
                      utah_ppg = "Utah"))

ggplot(bar_df, aes(x = player_name, y = ppg, fill = team)) +
  geom_col(position = "dodge") +
  labs(
    title = "Arizona vs Utah: Points Per Game by Player",
    x = "Player",
    y = "Points Per Game",
    fill = ""
  ) +
  scale_fill_manual(values = c("Arizona" = "#8B1A1A", "Utah" = "#5DADE2")) +
  theme_minimal() +
  theme(
    axis.text.x = element_text(angle = 45, hjust = 1)
  )

```



Percent change (ARI → UTAH)

```

plot1_df <- players_summary %>%
  drop_na(arizona_ppg, utah_ppg) %>%
  mutate(
    pct_change = (utah_ppg - arizona_ppg) / arizona_ppg * 100
  )

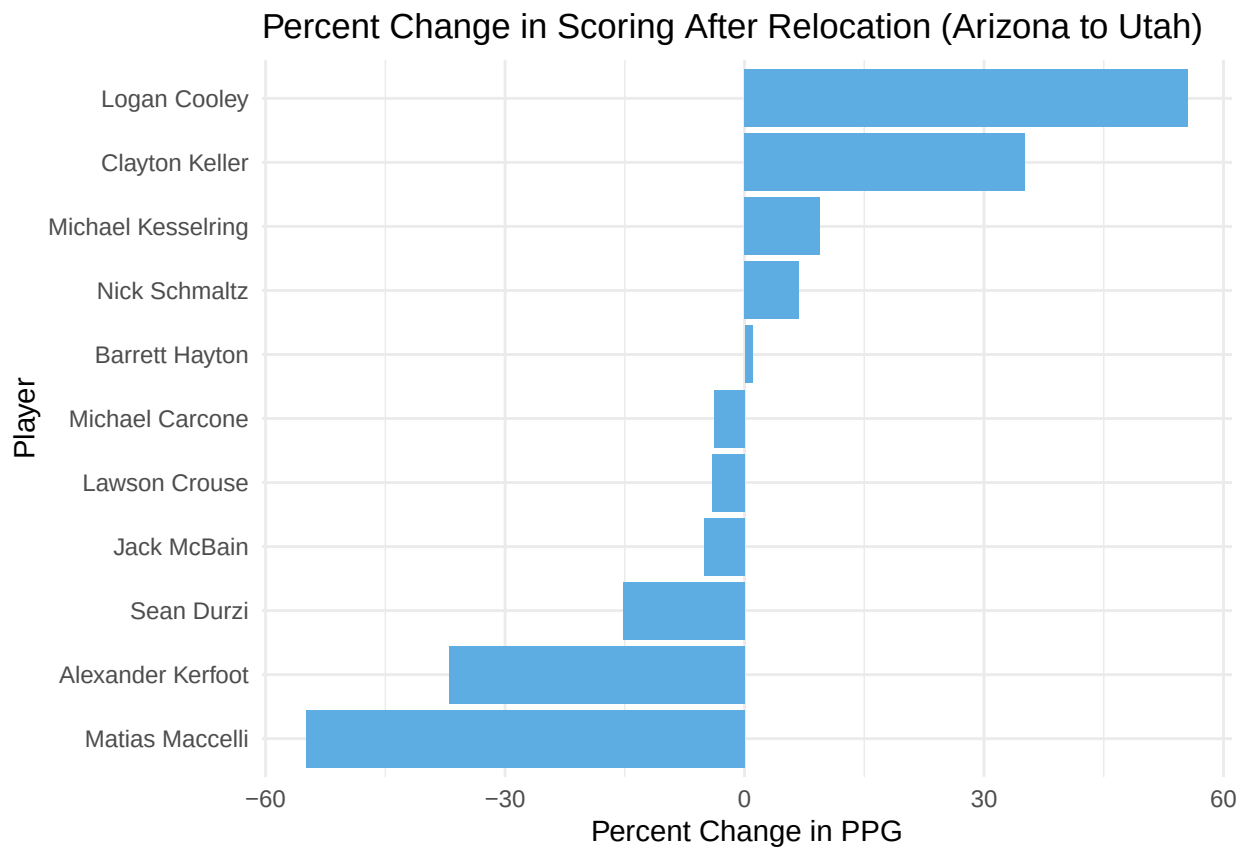
```

```

) %>%
  arrange(pct_change)

ggplot(plot1_df, aes(x = reorder(player_name, pct_change), y = pct_change)) +
  geom_col(fill = "#5DADE2") +
  coord_flip() +
  labs(
    title = "Percent Change in Scoring After Relocation (Arizona to Utah)",
    x = "Player",
    y = "Percent Change in PPG"
  ) +
  theme_minimal()

```



Creating years_in_system variable

```

joining_year <- c(
  "Clayton Keller"      = 2016,
  "Nick Schmaltz"       = 2019,
  "Lawson Crouse"       = 2016,
  "Michael Carcone"     = 2021,
  "Logan Cooley"        = 2023,
  "Sean Durzi"          = 2023,
  "Barrett Hayton"      = 2019,
  "Alexander Kerfoot"   = 2023,
  "Michael Kesselring"  = 2022,
  "Matias Maccelli"     = 2021,

```

```

"Jack McBain"      = 2021
)

# Extract the start year from the season string
df <- df %>%
  mutate(
    season_start = as.numeric(substr(season, 1, 4)),
    years_in_system = season_start - joining_year[player_name] + 1
  )

df %>%
  select(player_name, season, years_in_system) %>%
  arrange(player_name, season) %>%
  head(50)

```

##	player_name	season	years_in_system
## 1	Alexander Kerfoot	2023-2024	1
## 2	Alexander Kerfoot	2024-2025	2
## 3	Barrett Hayton	2021-2022	3
## 4	Barrett Hayton	2022-2023	4
## 5	Barrett Hayton	2024-2025	6
## 6	Barrett Hayton	2025-2026	7
## 7	Clayton Keller	2017-2018	2
## 8	Clayton Keller	2018-2019	3
## 9	Clayton Keller	2019-2020	4
## 10	Clayton Keller	2020-2021	5
## 11	Clayton Keller	2021-2022	6
## 12	Clayton Keller	2022-2023	7
## 13	Clayton Keller	2023-2024	8
## 14	Clayton Keller	2024-2025	9
## 15	Clayton Keller	2025-2026	10
## 16	Jack McBain	2022-2023	2
## 17	Jack McBain	2023-2024	3
## 18	Jack McBain	2024-2025	4
## 19	Jack McBain	2025-2026	5
## 20	Lawson Crouse	2016-2017	1
## 21	Lawson Crouse	2018-2019	3
## 22	Lawson Crouse	2019-2020	4
## 23	Lawson Crouse	2020-2021	5
## 24	Lawson Crouse	2021-2022	6
## 25	Lawson Crouse	2022-2023	7
## 26	Lawson Crouse	2023-2024	8
## 27	Lawson Crouse	2024-2025	9
## 28	Lawson Crouse	2025-2026	10
## 29	Logan Cooley	2023-2024	1
## 30	Logan Cooley	2024-2025	2
## 31	Logan Cooley	2025-2026	3
## 32	Matias Maccelli	2022-2023	2
## 33	Matias Maccelli	2023-2024	3
## 34	Matias Maccelli	2024-2025	4
## 35	Michael Carcone	2023-2024	3
## 36	Michael Carcone	2024-2025	4
## 37	Michael Carcone	2025-2026	5
## 38	Michael Kesselring	2023-2024	2

```
## 39 Michael Kesselring 2024-2025      3
## 40      Nick Schmaltz 2019-2020      1
## 41      Nick Schmaltz 2020-2021      2
## 42      Nick Schmaltz 2021-2022      3
## 43      Nick Schmaltz 2022-2023      4
## 44      Nick Schmaltz 2023-2024      5
## 45      Nick Schmaltz 2024-2025      6
## 46      Nick Schmaltz 2025-2026      7
## 47      Sean Durzi 2023-2024         1
## 48      Sean Durzi 2025-2026         3
```

Functions

```
# Activation functions
sigmoid <- function(z) 1 / (1 + exp(-z))
sigmoid_deriv <- function(z) {
  s <- sigmoid(z)
  s * (1 - s)
}
tanh_deriv <- function(z) 1 - tanh(z)^2

# Forward pass
forward_pass <- function(X, W1, b1, W2, b2,
                          activation = "tanh") {
  Z1 <- X %*% t(W1) +
    matrix(b1, nrow(X), length(b1), byrow = TRUE)
  if (activation == "sigmoid") {
    A1 <- sigmoid(Z1)
  } else {
    A1 <- tanh(Z1)
  }
  f <- A1 %*% W2 + b2
  list(Z1 = Z1, A1 = A1, f = f)
}

# Loss functions
mse_loss <- function(y, y_hat) mean((y - y_hat)^2)
r_squared <- function(y, y_hat) {
  1 - sum((y - y_hat)^2) / sum((y - mean(y))^2)
}

# Train function
train_nn <- function(X, y, K = 10, lr = 0.01,
                     epochs = 500, lambda = 0.01,
                     activation = "tanh",
                     batch_size = 10,
                     seed = 123) {
  set.seed(seed)
  n <- nrow(X)
  p <- ncol(X)
  W1 <- matrix(rnorm(K * p, sd = 0.1), K, p)
  b1 <- rep(0, K)
  W2 <- rnorm(K, sd = 0.1)
  b2 <- 0
```

```

loss_history <- numeric(epochs)
for (epoch in 1:epochs) {
  idx <- sample(n)
  X_shuf <- X[idx, , drop = FALSE]
  y_shuf <- y[idx]
  for (start in seq(1, n, by = batch_size)) {
    end <- min(start + batch_size - 1, n)
    X_batch <- X_shuf[start:end, , drop = FALSE]
    y_batch <- y_shuf[start:end]
    m <- nrow(X_batch)
    fp <- forward_pass(X_batch, W1, b1, W2, b2, activation)
    residual <- fp$f - y_batch
    dW2 <- (t(fp$A1) %*% residual) / m + 2 * lambda * W2
    db2 <- mean(residual)
    delta <- residual %*% t(W2)
    if (activation == "sigmoid") {
      delta <- delta * sigmoid_deriv(fp$Z1)
    } else {
      delta <- delta * tanh_deriv(fp$Z1)
    }
    dW1 <- (t(delta) %*% X_batch) / m + 2 * lambda * W1
    db1 <- colMeans(delta)
    W1 <- W1 - lr * dW1
    b1 <- b1 - lr * db1
    W2 <- W2 - lr * dW2
    b2 <- b2 - lr * db2
  }
  fp_full <- forward_pass(X, W1, b1, W2, b2, activation)
  loss_history[epoch] <- mse_loss(y, fp_full$f)
}
list(W1 = W1, b1 = b1, W2 = W2, b2 = b2,
     loss_history = loss_history)
}

```

Simulated Data: Why Linear Regression Fails

```

set.seed(42)
n_sim <- 100
x_sim <- seq(-pi, pi, length.out = n_sim)
y_sim <- sin(x_sim) + rnorm(n_sim, sd = 0.2)

sim_df <- data.frame(x = x_sim, y = y_sim)

ggplot(sim_df, aes(x = x, y = y)) +
  geom_point(alpha = 0.6, color = "gray40") +
  geom_smooth(method = "lm", color = "#8B1A1A",
             se = FALSE, linetype = "dashed") +
  labs(
    title = "Linear Regression: Sine Wave",
    subtitle = "A straight line cannot capture nonlinear structure",
    x = "X", y = "Y"
  ) +
  theme_minimal()

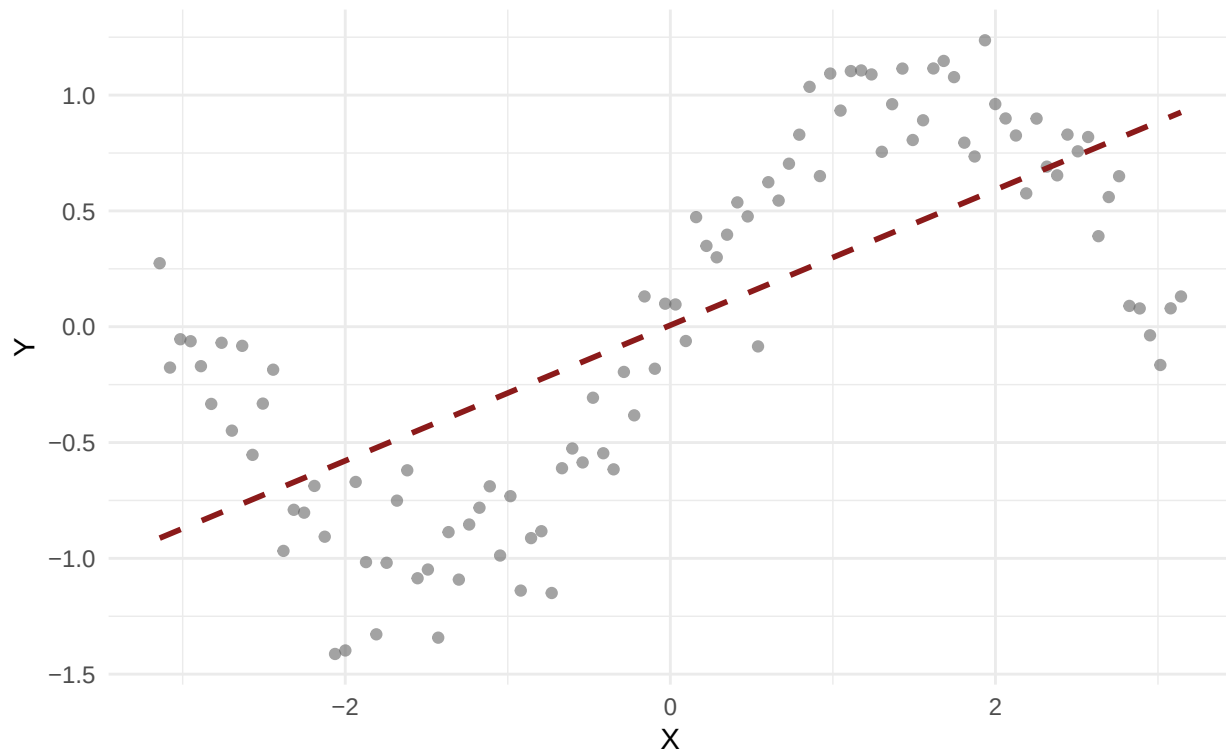
```



```
## `geom_smooth()` using formula = 'y ~ x'
```

Linear Regression: Sine Wave

A straight line cannot capture nonlinear structure



```
X_sim <- matrix(x_sim, ncol = 1)
X_sim_scaled <- scale(X_sim)
```

Effect of Number of Hidden Units (K)

```
k_values <- c(2, 5, 20)
results_sim <- list()

for (k in k_values) {
  model_sim <- train_nn(
    X_sim_scaled, y_sim,
    K = k,
    lr = 0.05,
    epochs = 1000,
    lambda = 0.001,
    activation = "tanh",
    batch_size = 20
  )
  fp_sim <- forward_pass(X_sim_scaled, model_sim$W1,
                        model_sim$b1, model_sim$W2, model_sim$b2)
  results_sim[[paste0("K=", k)]] <- as.numeric(fp_sim$f)
}
```

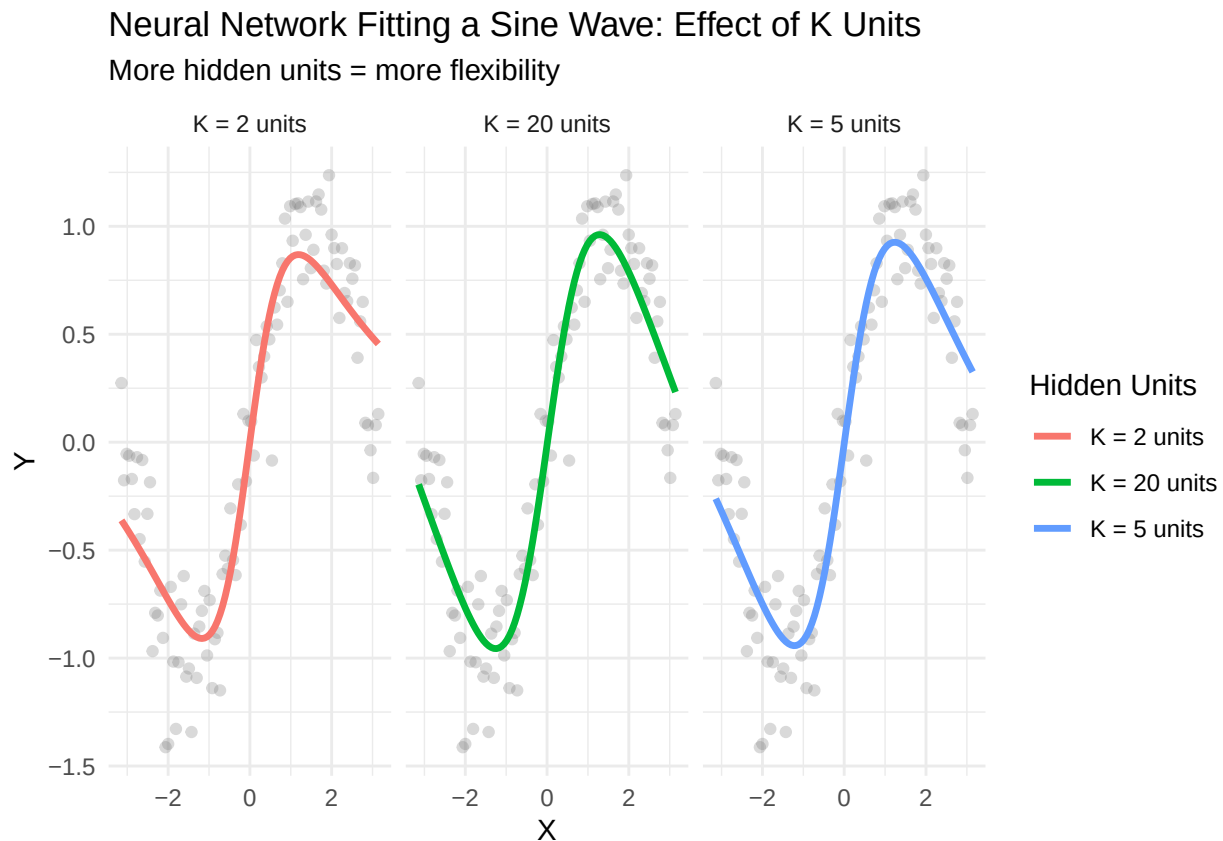
```
sim_plot_df <- data.frame(
  x = rep(x_sim, 3),
```

```

y_true = rep(y_sim, 3),
y_pred = c(results_sim[["K=2"]],
            results_sim[["K=5"]],
            results_sim[["K=20"]]),
K = rep(c("K = 2 units", "K = 5 units", "K = 20 units"),
        each = n_sim)
)

ggplot(sim_plot_df, aes(x = x)) +
  geom_point(aes(y = y_true), alpha = 0.3, color = "gray50") +
  geom_line(aes(y = y_pred, color = K), linewidth = 1.2) +
  labs(
    title = "Neural Network Fitting a Sine Wave: Effect of K Units",
    subtitle = "More hidden units = more flexibility",
    x = "X", y = "Y",
    color = "Hidden Units"
  ) +
  theme_minimal() +
  facet_wrap(~K)

```



Sigmoid vs Tanh Activation Functions

```

activations <- c("sigmoid", "tanh")
act_results <- list()

for (act in activations) {

```

```

model_act <- train_nn(
  X_sim_scaled, y_sim,
  K = 5,
  lr = 0.05,
  epochs = 1000,
  lambda = 0.001,
  activation = act,
  batch_size = 20
)
fp_act <- forward_pass(X_sim_scaled, model_act$W1,
  model_act$b1, model_act$W2, model_act$b2,
  activation = act)
act_results[[act]] <- as.numeric(fp_act$f)
}

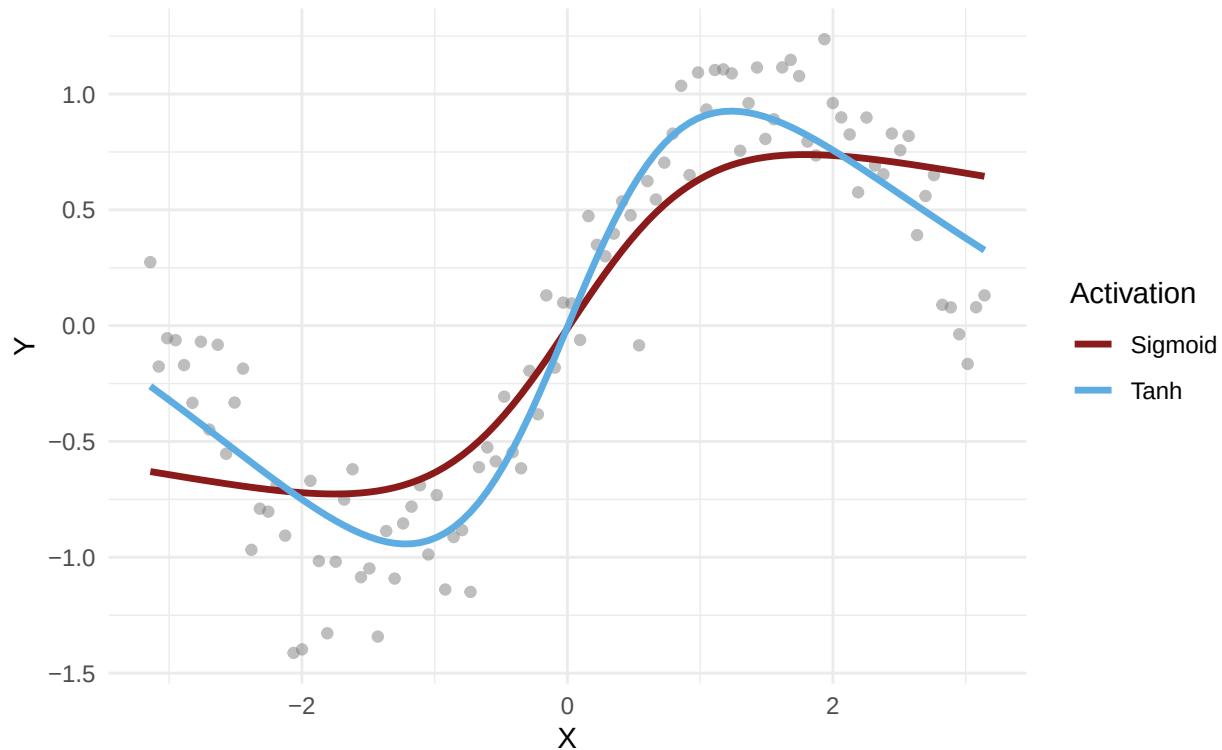
act_plot_df <- data.frame(
  x = rep(x_sim, 2),
  y_true = rep(y_sim, 2),
  y_pred = c(act_results[["sigmoid"]],
    act_results[["tanh"]]),
  activation = rep(c("Sigmoid", "Tanh"), each = n_sim)
)

ggplot(act_plot_df, aes(x = x)) +
  geom_point(aes(y = y_true), alpha = 0.3, color = "gray50") +
  geom_line(aes(y = y_pred, color = activation), linewidth = 1.2) +
  scale_color_manual(values = c("Sigmoid" = "#8B1A1A",
    "Tanh" = "#5DADE2")) +
  labs(
    title = "Sigmoid vs Tanh Activation Function",
    subtitle = "K = 5 units, comparing two activation functions",
    x = "X", y = "Y",
    color = "Activation"
  ) +
  theme_minimal()

```

Sigmoid vs Tanh Activation Function

K = 5 units, comparing two activation functions



Effect of L2 Regularization

```
lambdas <- c(0.0001, 0.01, 0.5)
reg_results <- list()

for (lam in lambdas) {
  model_reg <- train_nn(
    X_sim_scaled, y_sim,
    K = 20,
    lr = 0.05,
    epochs = 1000,
    lambda = lam,
    activation = "tanh",
    batch_size = 20
  )
  fp_reg <- forward_pass(X_sim_scaled, model_reg$W1,
                        model_reg$b1, model_reg$W2, model_reg$b2)
  reg_results[[paste0("lambda=", lam)]] <- as.numeric(fp_reg$f)
}

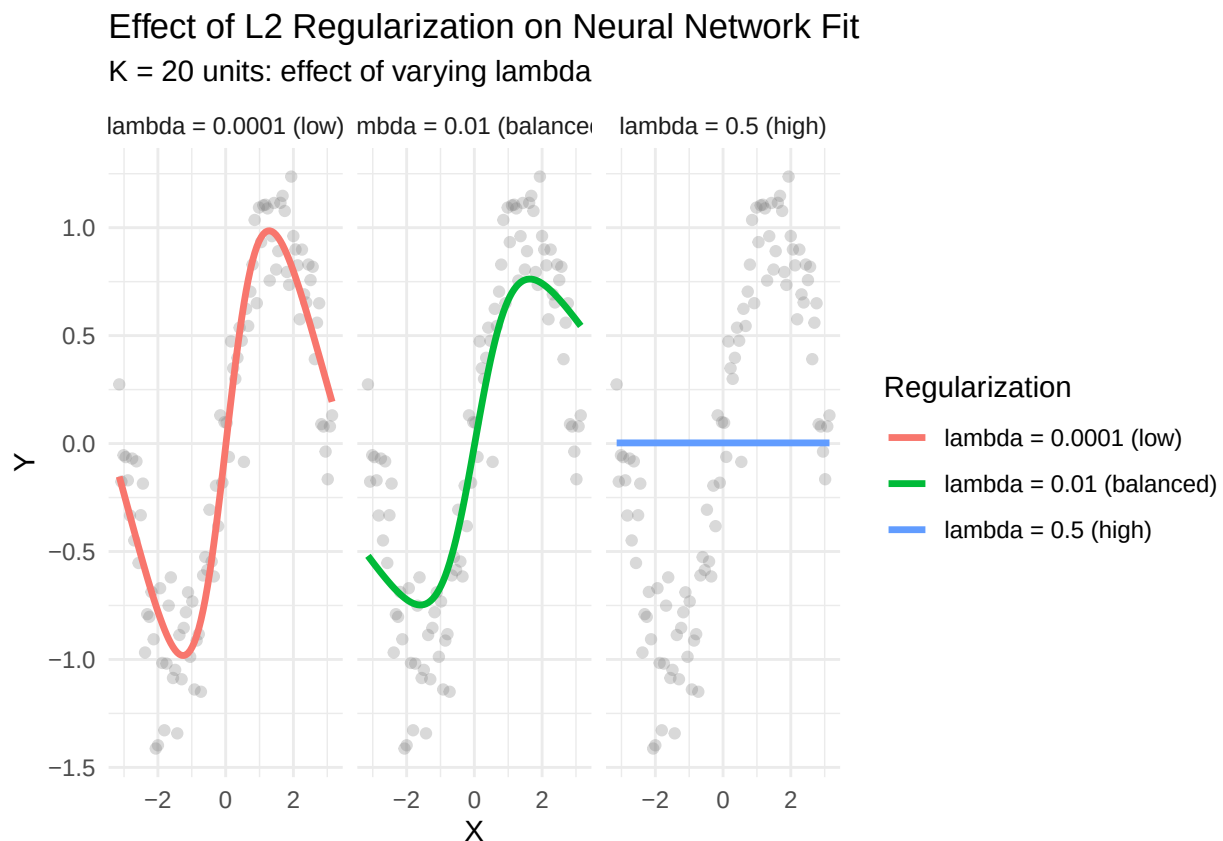
reg_plot_df <- data.frame(
  x = rep(x_sim, 3),
  y_true = rep(y_sim, 3),
  y_pred = c(reg_results[["lambda=1e-04"]],
             reg_results[["lambda=0.01"]],
             reg_results[["lambda=0.5"]]),
)
```

```

lambda = rep(c("lambda = 0.0001 (low)",
               "lambda = 0.01 (balanced)",
               "lambda = 0.5 (high)"), each = n_sim)
)

ggplot(reg_plot_df, aes(x = x)) +
  geom_point(aes(y = y_true), alpha = 0.3, color = "gray50") +
  geom_line(aes(y = y_pred, color = lambda), linewidth = 1.2) +
  labs(
    title = "Effect of L2 Regularization on Neural Network Fit",
    subtitle = "K = 20 units: effect of varying lambda",
    x = "X", y = "Y",
    color = "Regularization"
  ) +
  theme_minimal() +
  facet_wrap(~lambda)

```



Effect of Learning Rate

```

learning_rates <- c(0.001, 0.01, 0.1)
lr_results <- list()

for (lr in learning_rates) {
  model_lr <- train_nn(
    X_sim_scaled, y_sim,
    K = 5,

```

```

    lr = lr,
    epochs = 1000,
    lambda = 0.001,
    activation = "tanh",
    batch_size = 20
  )
  fp_lr <- forward_pass(X_sim_scaled, model_lr$W1,
                        model_lr$b1, model_lr$W2, model_lr$b2)
  lr_results[[paste0("lr=", lr)]] <- as.numeric(fp_lr$f)
}

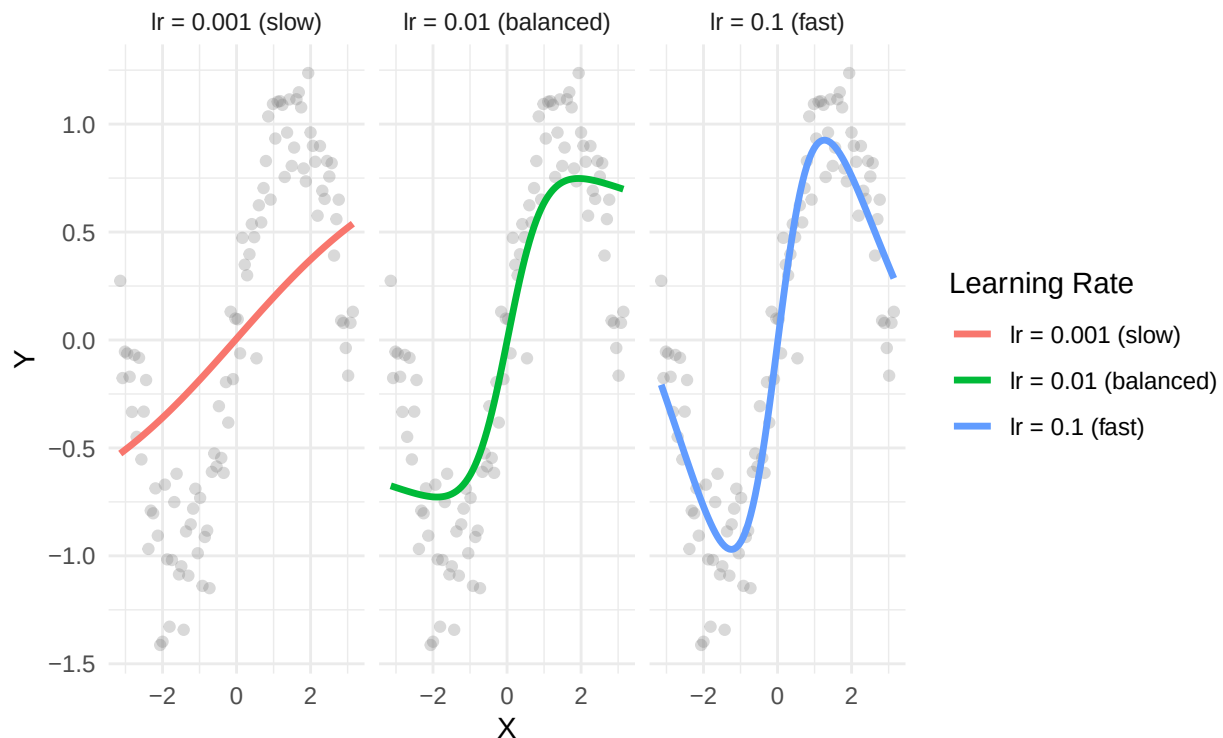
lr_plot_df <- data.frame(
  x = rep(x_sim, 3),
  y_true = rep(y_sim, 3),
  y_pred = c(lr_results[["lr=0.001"]],
              lr_results[["lr=0.01"]],
              lr_results[["lr=0.1"]]),
  lr = rep(c("lr = 0.001 (slow)",
              "lr = 0.01 (balanced)",
              "lr = 0.1 (fast)"), each = n_sim)
)

ggplot(lr_plot_df, aes(x = x)) +
  geom_point(aes(y = y_true), alpha = 0.3, color = "gray50") +
  geom_line(aes(y = y_pred, color = lr), linewidth = 1.2) +
  labs(
    title = "Effect of Learning Rate on Neural Network Fit",
    subtitle = "K = 5 units, varying learning rate",
    x = "X", y = "Y",
    color = "Learning Rate"
  ) +
  theme_minimal() +
  facet_wrap(~lr)

```

Effect of Learning Rate on Neural Network Fit

K = 5 units, varying learning rate



Neural network code

```
# -----
# 1. Prepare Data
# -----

df <- df %>%
  mutate(season_start = as.numeric(substr(season, 1, 4)))

nn_df <- df %>%
  arrange(player_name, season_start) %>%
  group_by(player_name) %>%

  mutate(prior_ppg_raw = lag(ppg)) %>%

  mutate(
    prior_ppg = ifelse(is.na(prior_ppg_raw), ppg, prior_ppg_raw)
  ) %>%

  ungroup() %>%
  select(player_name, team, season, ppg, age,
         years_in_system, avg_toi, prior_ppg)

# -----
# 2. Train/Test Split
# -----
```

```

set.seed(123)
train_idx <- sample(seq_len(nrow(nn_df)),
                    size = 0.8 * nrow(nn_df))

train_df <- nn_df[train_idx, ]
test_df  <- nn_df[-train_idx, ]

X_train <- train_df %>%
  select(age, years_in_system, avg_toi, prior_ppg) %>%
  as.matrix()

y_train <- train_df$ppg

X_test <- test_df %>%
  select(age, years_in_system, avg_toi, prior_ppg) %>%
  as.matrix()

y_test <- test_df$ppg

# -----
# 3. Scale Features
# -----

X_scaled_train <- scale(X_train)
X_mean <- attr(X_scaled_train, "scaled:center")
X_sd <- attr(X_scaled_train, "scaled:scale")

X_scaled_test <- scale(X_test, center = X_mean, scale = X_sd)

# -----
# 4. Train Model
# -----

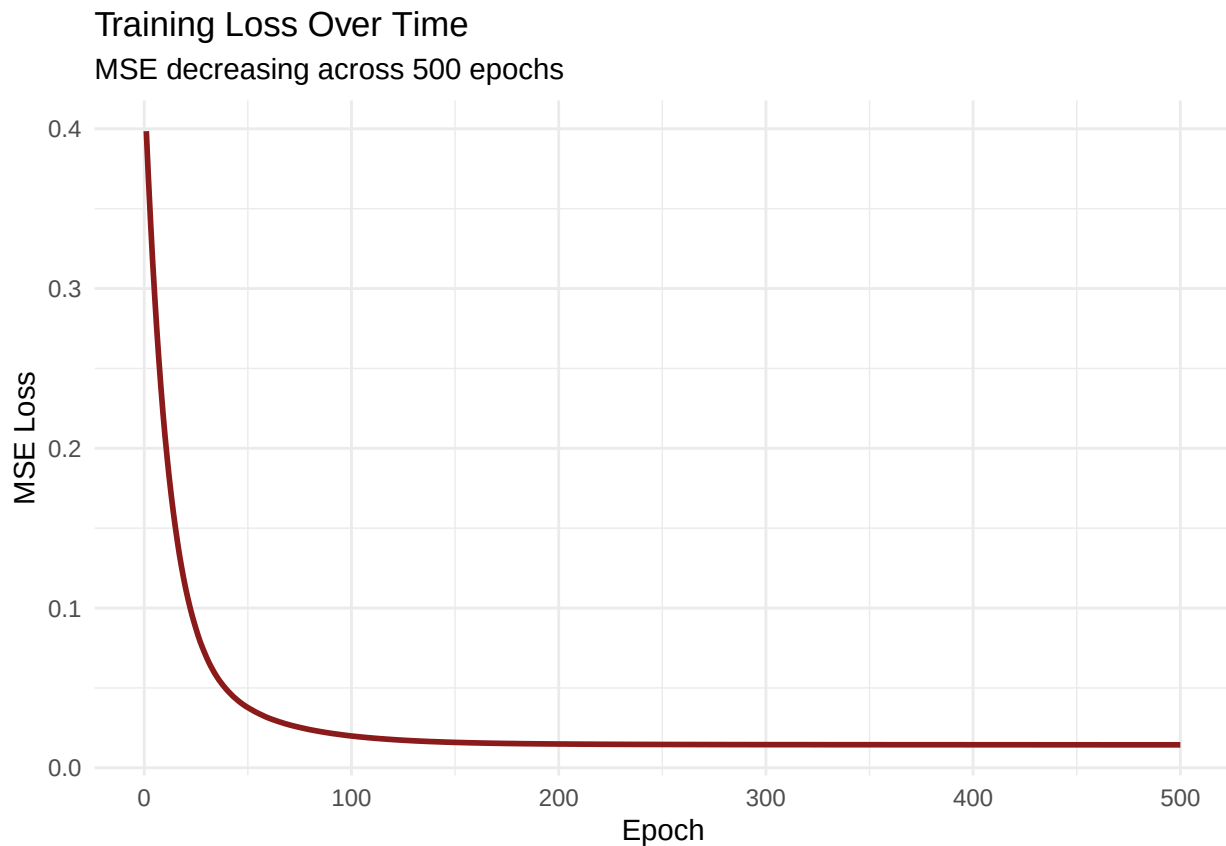
model <- train_nn(
  X_scaled_train, y_train,
  K = 10,
  lr = 0.01,
  epochs = 500,
  lambda = 0.01,
  activation = "tanh",
  batch_size = 10
)

# Plot training loss curve
data.frame(
  epoch = 1:length(model$loss_history),
  loss = model$loss_history
) %>%
  ggplot(aes(x = epoch, y = loss)) +
  geom_line(color = "#8B1A1A", linewidth = 1) +
  labs(
    title = "Training Loss Over Time",
    subtitle = "MSE decreasing across 500 epochs",
  )

```



```
x = "Epoch",
y = "MSE Loss"
) +
theme_minimal()
```



```
# -----
# 5. Predictions
# -----

fp_train <- forward_pass(X_scaled_train, model$W1, model$b1,
                        model$W2, model$b2)

fp_test <- forward_pass(X_scaled_test, model$W1, model$b1,
                       model$W2, model$b2)

train_df$predicted_ppg <- as.numeric(fp_train$f)
test_df$predicted_ppg <- as.numeric(fp_test$f)

train_df$residual <- train_df$ppg - train_df$predicted_ppg
test_df$residual <- test_df$ppg - test_df$predicted_ppg

nn_results <- bind_rows(train_df, test_df)

# -----
# 6. Evaluation
# -----
```

```
cat("Train MSE:", mse_loss(y_train, train_df$predicted_ppg), "\n")
```

```
## Train MSE: 0.01437413
```

```
cat("Test MSE:", mse_loss(y_test, test_df$predicted_ppg), "\n")
```

```
## Test MSE: 0.01667668
```

```
cat("Train R2:", r_squared(y_train, train_df$predicted_ppg), "\n")
```

```
## Train R2: 0.7714932
```

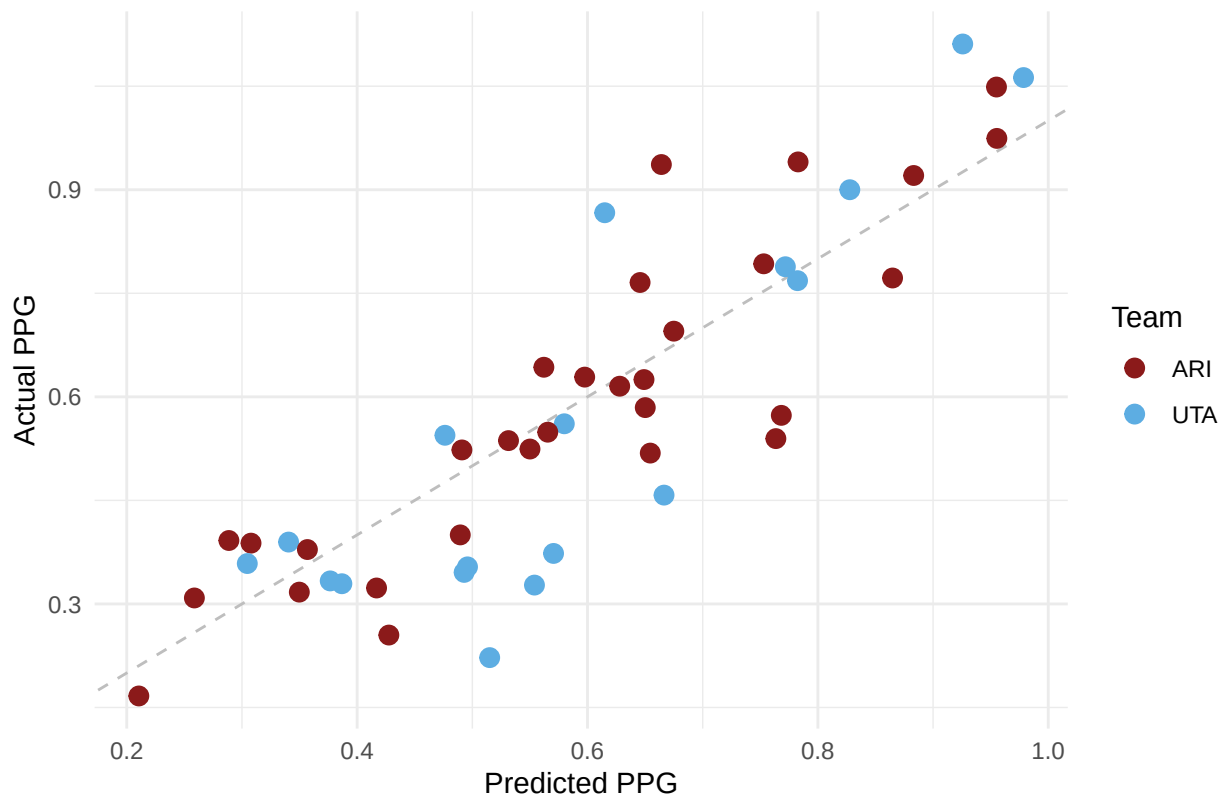
```
cat("Test R2:", r_squared(y_test, test_df$predicted_ppg), "\n")
```

```
## Test R2: 0.559515
```

```
# -----  
# 7. Plots  
# -----
```

```
ggplot(nn_results,  
  aes(x = predicted_ppg,  
       y = ppg,  
       color = team)) +  
  geom_abline(slope = 1, intercept = 0,  
             linetype = "dashed", color = "gray") +  
  geom_point(size = 3) +  
  scale_color_manual(values = c(  
    "ARI" = "#8B1A1A",  
    "UTA" = "#5DADE2"  
  )) +  
  labs(  
    title = "Actual vs Expected PPG (Neural Network Model)",  
    x = "Predicted PPG",  
    y = "Actual PPG",  
    color = "Team"  
  ) +  
  theme_minimal()
```

Actual vs Expected PPG (Neural Network Model)



```
# -----
# Plot: color = team, shape = player
# -----

focus_players <- c("Clayton Keller", "Lawson Crouse", "Logan Cooley")

plot_df <- nn_results %>%
  filter(player_name %in% focus_players)

ggplot(plot_df,
  aes(x = predicted_ppg,
    y = ppg,
    color = team,
    shape = player_name)) +

  geom_abline(slope = 1, intercept = 0,
    linetype = "dashed",
    color = "gray") +

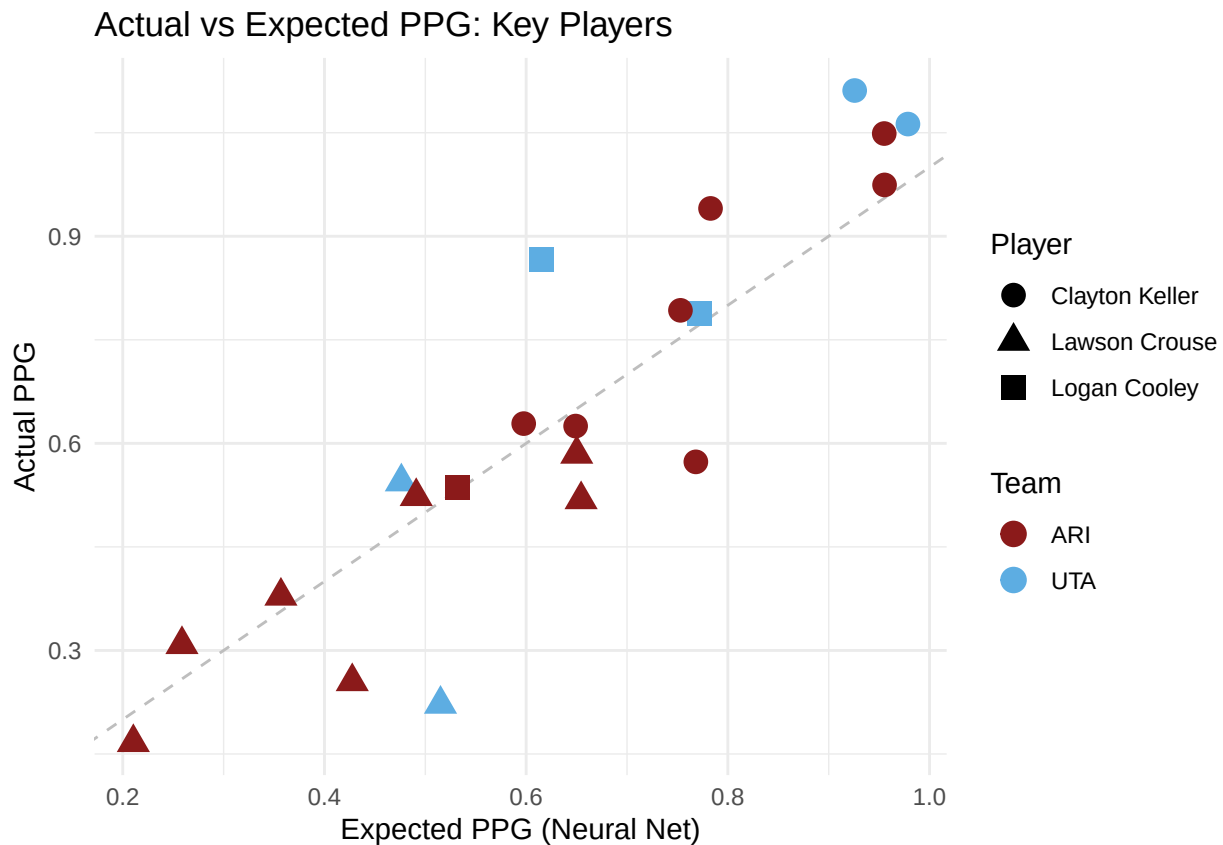
  geom_point(size = 4) +

  # Team colors (ARI vs UTA)
  scale_color_manual(values = c(
    "ARI" = "#8B1A1A", # deep red
    "UTA" = "#5DADE2" # light blue
  )) +
```

```
# Player shapes
scale_shape_manual(values = c(
  "Clayton Keller" = 16,
  "Lawson Crouse" = 17,
  "Logan Cooley" = 15
)) +

labs(
  title = "Actual vs Expected PPG: Key Players",
  x = "Expected PPG (Neural Net)",
  y = "Actual PPG",
  color = "Team",
  shape = "Player"
) +

theme_minimal()
```



Model Residuals by Team

```
ggplot(nn_results, aes(x = team, y = residual, fill = team)) +
  geom_boxplot(alpha = 0.7) +
  geom_hline(yintercept = 0, linetype = "dashed", color = "gray") +
  scale_fill_manual(values = c(
    "ARI" = "#8B1A1A",
    "UTA" = "#5DADE2"
  )) +
```

```
labs(
  title = "Model Residuals by Team",
  subtitle = "Positive residual = outperformed expectations",
  x = "Team",
  y = "Residual (Actual - Predicted PPG)"
) +
theme_minimal() +
theme(legend.position = "none")
```

